p :	q :	r :
`write x; (1)`	`read y;` `read x; (2)`	`read z;` `read x; (3)`

Fig. 1. Writer-readers code excerpt.

1991]. For instance, we allow a send transition to be dependent with another send transition only if the order in which the two messages are received is significant. Similarly to preceding papers on DPOR, we assume, for simplicity of exposition, that threads are deterministic. Our techniques can be extended to control nondeterminism without introducing fundamentally new concepts, whereas data nondeterminism typically requires additional symbolic techniques.

We have implemented both *source-DPOR* and *optimal-DPOR* as extensions for Concuerror [Christakis et al. 2013], a stateless model checking tool for Erlang programs. Erlang's concurrency model focuses primarily on message passing, but it is also possible to write programs which manipulate shared data structures. Our evaluation shows that on a wide selection of benchmarks, including benchmarks from the DPOR literature, but more importantly on real Erlang applications of considerable size, we obtain optimal or very close to optimal reduction in the number of interleavings even with *source-DPOR*, therefore significantly outperforming the original DPOR algorithm not only in number of explored interleavings but in total execution time as well.

Organization. In the next section, we illustrate the basic new ideas of our techniques. We introduce our computational model and formulation of the partial-order framework in Section 3. To simplify presentation, we initially assume that processes do not disable each other. In Section 4 we introduce source sets, and establish that source sets are necessary and sufficient for soundness of any DPOR algorithm. The *source-DPOR* algorithm is described in Section 5. We formalize the concept of wakeup trees in Section 6, before describing the *optimal-DPOR* algorithm in Section 7. We then extend the algorithms to computational models where processes may disable each other, e.g., via locks, in Section 8. Section 9 discusses some trade-offs in computational cost between source-DPOR and optimal-DPOR. Implementation of the algorithms is described in Section 10, and experimental evaluation in Section 11. The article ends by surveying related work and offering some concluding remarks.

2 BASIC IDEAS

In this section, we give an informal introduction to the concepts of source sets and wakeup trees, and their improvement over existing approaches, using some small examples.

Source Sets. In Fig. 1, the three processes p, q, and r perform dependent accesses to the shared variable `x`. In this example, let us consider two accesses as dependent if they access the same variable and one of them is a write. Since there are no writes to `y` and `z` here, the accesses to `y` and `z` are not dependent with anything else. For this program, there are four Mazurkiewicz traces (i.e., equivalence classes of executions), each characterized by its sequence of accesses to `x` (three accesses can be ordered in six ways, but two pairs of orderings are equivalent since they differ only in the ordering of adjacent reads, which are not dependent).

Any POR method selects some subset of $\{p, q, r\}$ to perform some first step in the set of explored executions. It is not enough to select only p, since then executions where some read access happens before the write access of p will not be explored. In DPOR, assume that the first execution to be explored is $p.q.q.r.r$ (we denote executions by the dotted sequence of scheduled process steps). A